

MODULE 3 — PROJECT 2

LearningSteps Evolution

From Manual Deployment to Automated DevSecOps

Capstone Project Report

GITHUB address:

<https://github.com/Hany-Dh/learningsteps>

Field	Detail
Prepared by	Hany Dhannoon (HAGA)
Course	Cybersecurity Fundamentals — Applied DevSecOps Track
Project	Module 3, Project 2 — LearningSteps Evolution
Repository	github.com/Hany-Dh/learningsteps (forked from CyberstepsDE/learningsteps)
Cloud Platform	Microsoft Azure (Resource Group: module3-project2, Region: West Europe)
Report Date	October 2026
Classification	Internal / Academic Submission

Hany Dhannoon

06.10.2026

Table of Contents

1. Executive Summary.....	3
2. Project Objectives and Scope	4
2.1 Mission	4
2.2 In-Scope Deliverables	4
2.3 Success Criteria (as defined by the project brief)	4
3. Target Architecture	5
3.1 Component Summary	5
4. Methodology.....	6
5. Phase-by-Phase Implementation.....	9
5.0 Phase 0 — Plan & Architecture	9
5.1 Phase 1 — Install Tools (Kali Linux)	9
5.2 Phase 2 — Get the Application Working Locally.....	9
Verified endpoint status after repair.....	10
5.3 Phase 3 — Containerize and Push to ACR.....	10
Vulnerability scan result	10
5.4 Phase 4 — Provision Infrastructure with Terraform.....	11
Final resource configuration.....	11
Infrastructure security scan.....	11
5.5 Phase 5 — Connect Key Vault Secrets	11
5.6 Phase 6 — Deploy with Helm Chart	12
Checkpoint verification.....	12
5.7 Phase 7 — CI/CD Pipeline (OIDC + Security Gates)	12
Pipeline stage summary.....	12
5.8 Phase 8 — Optional Monitoring (Prometheus & Grafana)	13
Completed	13
Remaining (not completed).....	13
Resource sizing decision	13
5.9 Phase 9 — Final Demonstration & Handover	14
6. Feature Enhancement — Dual Content-Type Support.....	14
Implementation	14
Verification	14
7. Consolidated Troubleshooting and Issue Log	15
8. Security Posture Summary.....	17
8.1 Application Image.....	17
8.2 Infrastructure (Terraform)	17
8.3 Secrets Management	17
8.4 CI/CD Pipeline Gates	17

9. Observability Status	17
10. Final Demonstration Plan.....	18
10.1 Security Enforcement — Prove the Pipeline Blocks a Bad Push.....	18
10.2 Infrastructure Recovery — terraform destroy → terraform apply.....	18
11. Conclusion	19
Appendix A — Final Terraform Configuration (main.tf, abridged)	Fehler! Textmarke nicht definiert.
Appendix B — GitHub Actions Workflow (ci-cd.yml, abridged)	21
Appendix C — Helm Chart Structure	21
Appendix D — Tool and Platform Reference	22
Appendix E — Project Progress Tracker (Summary).....	22
Index of Figures and Tables	23

1. Executive Summary

This report documents the design, implementation, and verification of the LearningSteps Evolution capstone project — the transformation of a manually-provisioned, two-tier Azure application into a fully automated, security-gated DevSecOps delivery pipeline. The original LearningSteps API (a Python FastAPI journal application backed by PostgreSQL) had previously been deployed by hand: virtual machines clicked into existence through the Azure Portal, a database installed and configured manually, and application code shipped over SSH.

This project replaces every one of those manual steps with code. Infrastructure is defined declaratively in Terraform. The application is packaged as a container image and scanned for vulnerabilities before release. Kubernetes, via Azure Kubernetes Service (AKS), orchestrates the running application and keeps it self-healing. A GitHub Actions pipeline — authenticated to Azure through passwordless OIDC federation — builds, scans, and deploys the application automatically on every push to the main branch. Secrets are never stored in source control; they are held in Azure Key Vault and delivered to the running pod through the Secrets Store CSI Driver.

All nine required project phases were completed and independently verified against the live Azure environment. An optional tenth phase — Prometheus and Grafana observability — was substantially implemented (metrics instrumentation, scraping, and dashboard infrastructure all confirmed working) and then deliberately paused once its infrastructure-level goal was proven, since the project brief marks this phase optional and the remaining work was UI configuration rather than engineering risk.

Beyond the baseline requirements, the application programming interface was extended during the project to accept both application/json and text/plain request bodies on its write endpoints, broadening client compatibility without altering the underlying data model or storage logic.

The sections that follow document the full technical journey: the target architecture, the methodology and tooling, a phase-by-phase account of what was built and how it was verified, a consolidated log of every real issue encountered and its resolution, the security posture of the final system, and a forward plan for the two remaining handover demonstrations (pipeline security enforcement and infrastructure recovery), which are documented as a ready-to-execute plan rather than run against the live environment, to avoid tearing down a working demo ahead of presentation.

2. Project Objectives and Scope

2.1 Mission

Evolve the existing LearningSteps API from a hand-built, static deployment into a self-healing, automated, and observable ecosystem, without modifying the original upstream application logic beyond what is required for operability.

2.2 In-Scope Deliverables

- A production-oriented Dockerfile for the FastAPI application.
- Terraform configuration provisioning all required Azure infrastructure (networking, compute, data, security).
- A DevSecOps-enabled CI/CD pipeline in GitHub Actions: build, lint, test, secret scan, vulnerability scan, infrastructure scan, OIDC-authenticated deploy.
- Kubernetes orchestration of the application on Azure Kubernetes Service (AKS), packaged as a Helm chart.
- Secrets management via Azure Key Vault, delivered into pods through the CSI Secrets Store driver — no secret ever committed to source control.
- (Optional) Observability via Prometheus metrics collection and Grafana dashboards.

2.3 Success Criteria (as defined by the project brief)

#	Criterion	Status
1	Continuous Deployment — a code push is live on the public IP automatically	Achieved and verified
2	Security Enforcement — pipeline fails on a High/Critical finding or a leaked secret	Gate implemented and active; live block-and-revert demo documented as a ready plan (Section 10)
3	Infrastructure Recovery — terraform destroy followed by terraform apply rebuilds the environment	Infrastructure is fully Terraform-managed and import-reconciled; full destroy/apply cycle documented as a ready plan (Section 10)
4	(Optional) Visual Insights — a live Grafana dashboard populated with real metrics	Infrastructure complete and scrape verified; dashboard panel configuration paused (optional item)

3. Target Architecture

The system follows a GitOps-adjacent delivery model: source changes in GitHub trigger an automated pipeline that builds and scans a container image, pushes it to a private registry, and updates the Kubernetes deployment. Infrastructure itself is owned by Terraform; application delivery is owned by the CI/CD pipeline; the two are deliberately kept separate so that an application release never requires a Terraform run, and an infrastructure change never requires rebuilding the application.

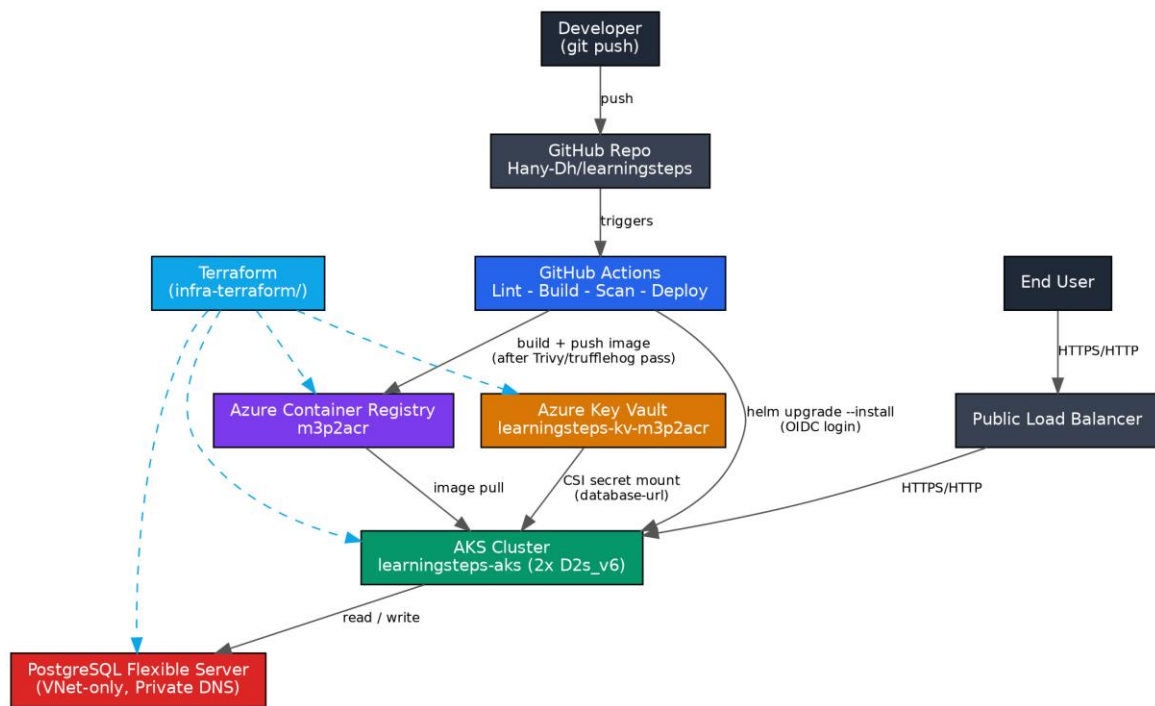


Figure 1.1 — End-to-end system architecture: source control, CI/CD, container registry, AKS, Key Vault, and PostgreSQL.

3.1 Component Summary

Component	Purpose
Virtual Network + Subnets	Isolated address space with dedicated subnets for AKS nodes and the database
Azure Kubernetes Service (AKS)	Runs the application as a self-healing, autoscaling workload
Azure Container Registry (ACR)	Stores built, vulnerability-scanned container images
Azure Database for PostgreSQL — Flexible Server	Private, VNet-only data store for journal entries
Azure Key Vault	Holds the database connection secret; delivered to pods via CSI driver, never stored in Git
Private DNS Zone	Required for VNet-integrated PostgreSQL name resolution
GitHub Actions	Executes the build–scan–push–deploy pipeline on every push to main
Azure AD App Registration (OIDC)	Passwordless federated identity GitHub Actions authenticates as — no stored Azure credential

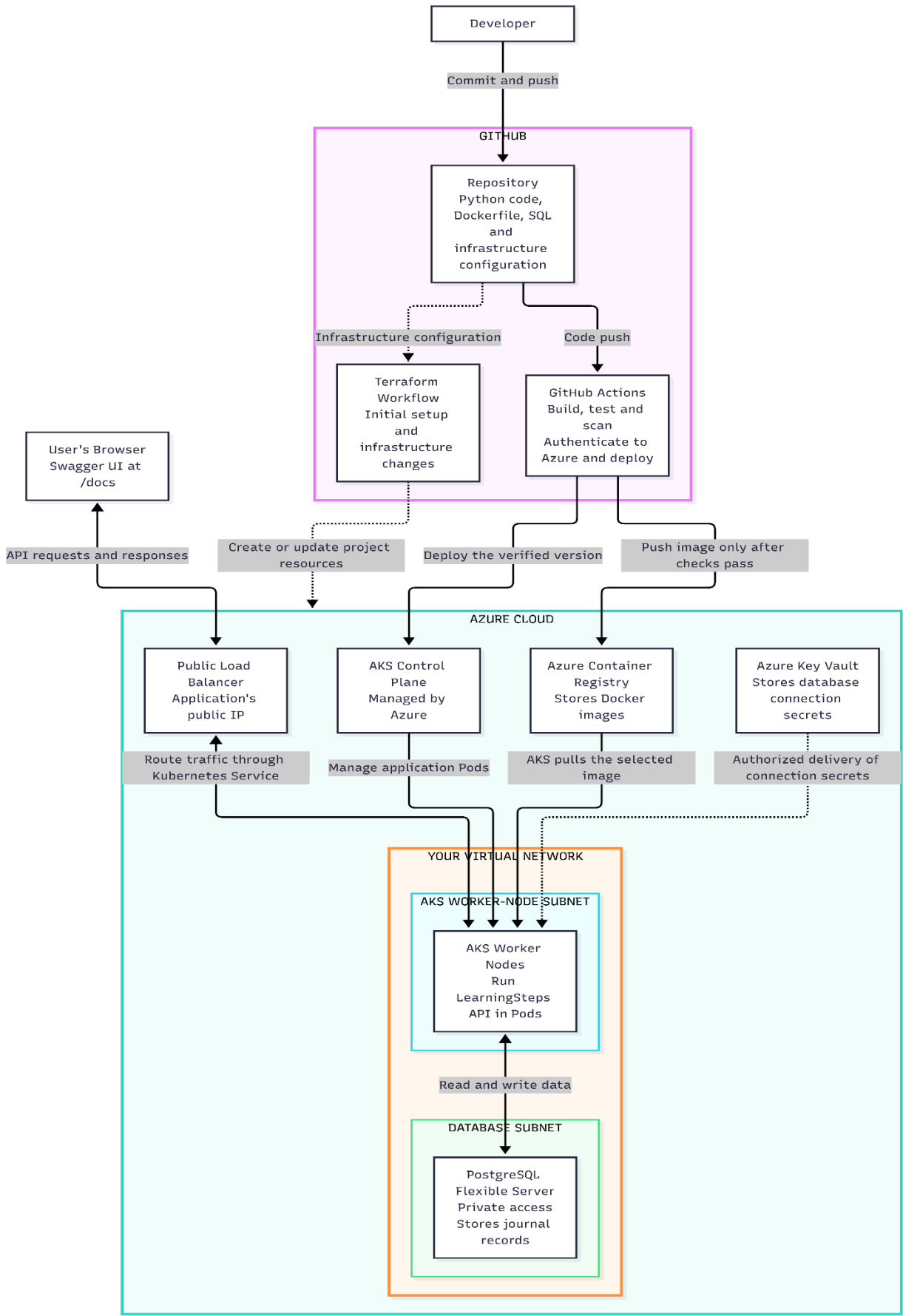


Figure 1.2

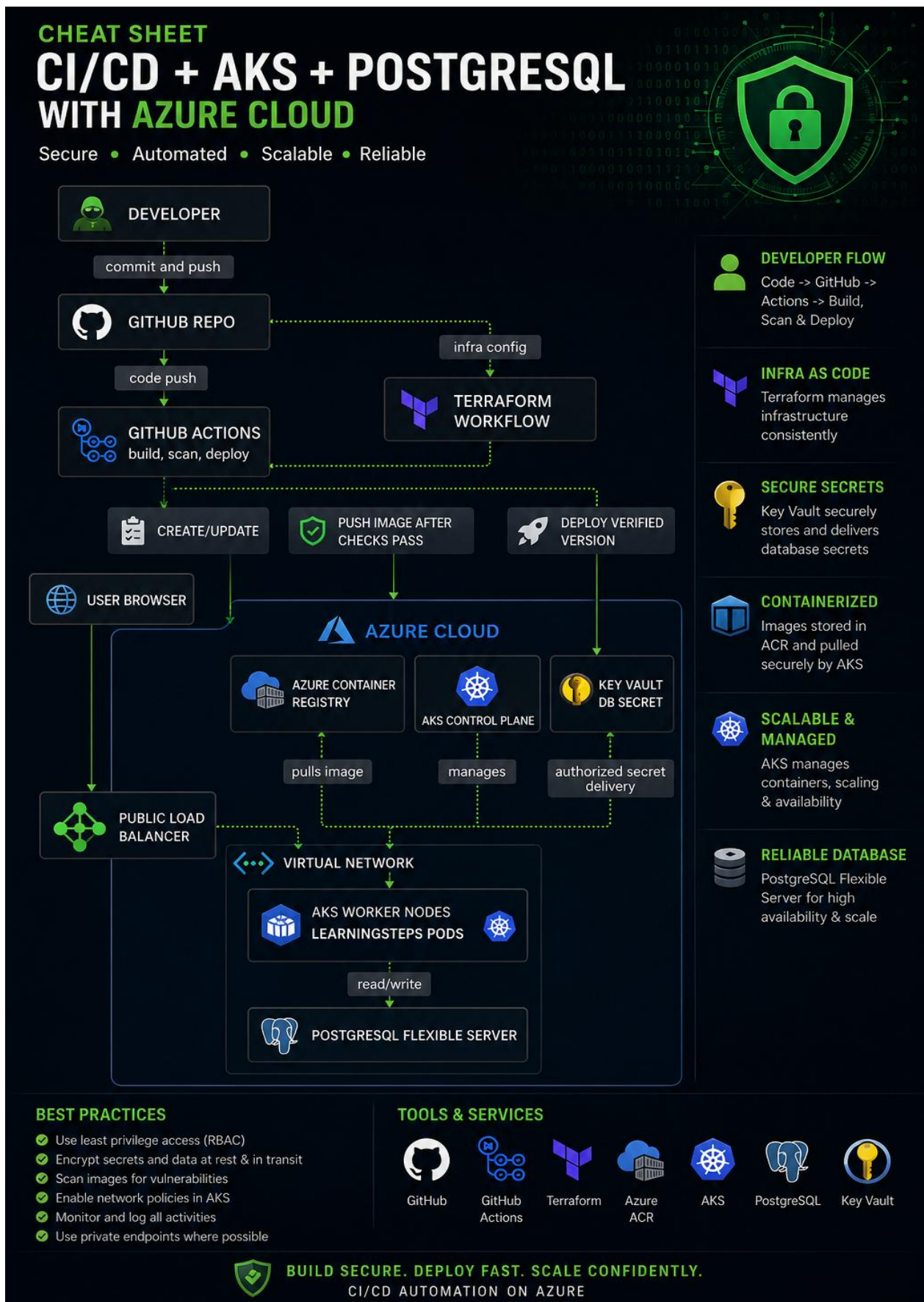


Figure 1.3

4. Methodology

The project was executed as ten sequential phases, each with a defined goal and an explicit checkpoint that had to be independently verified against the live environment before the next phase began. This mirrors standard DevSecOps delivery practice: infrastructure, security, and application concerns are built up incrementally rather than attempted all at once.



Figure 2 — The ten project phases, in execution order.

Phase	Title	Status
0	Plan & Architecture	Complete
1	Install Tools (Kali Linux workstation)	Complete
2	Get the Application Working Locally	Complete
3	Containerize and Push to ACR	Complete
4	Provision Infrastructure with Terraform	Complete
5	Connect Key Vault Secrets	Complete
6	Deploy with Helm Chart	Complete
7	CI/CD Pipeline (OIDC + Security Gates)	Complete
8	Optional Monitoring (Prometheus + Grafana)	Paused — infrastructure complete, optional per brief
9	Final Demonstration & Handover	Documented plan; execution reserved for live presentation

5. Phase-by-Phase Implementation

5.0 Phase 0 — Plan & Architecture

The project brief, the original LearningSteps repository, and the target architecture diagram supplied by the course were reviewed. The ten-phase execution plan in Section 4 was produced, and a living Notion workspace was established as the single source of truth for progress tracking, command references, and an issue log — updated continuously throughout the project and used as the primary reference for this report.

5.1 Phase 1 — Install Tools (Kali Linux)

All tooling was installed and verified on a Kali Linux workstation: Docker, Git, Python 3, Azure CLI, Terraform, kubectl, Helm, Trivy, and trufflehog.

Tool	Purpose
Docker	Build, run, and test the application container
Azure CLI	Create and manage Azure resources
Terraform	Provision infrastructure as code
kubectl	Interact with the AKS cluster
Helm	Package and deploy the application to Kubernetes
Trivy	Vulnerability scanning for images and Terraform configuration
trufflehog	Secret scanning in source and commit history

Two environment-specific issues were resolved during setup, documented fully in Section 7 (Consolidated Issue Log): the Azure CLI apt package is unavailable for Kali's rolling-release codename (resolved via pipx installation), and tfsec — originally planned for Terraform scanning — was found to be deprecated and merged into Trivy; all Terraform scanning in this project therefore uses trivy config instead.

5.2 Phase 2 — Get the Application Working Locally

The LearningSteps repository was cloned and its structure inspected: the FastAPI application lives under `api/`, with dependencies in `api/requirements.txt`, a PostgreSQL schema in `database_setup.sql`, and a six-endpoint CRUD API (POST, GET, GET-by-id, PATCH, DELETE, DELETE-all on `/entries`).

Local verification uncovered two intentionally unimplemented endpoints in the exercise code — `GET /entries/{id}` and `DELETE /entries/{id}` — both returning HTTP 501. Both were implemented (wiring already-correct service-layer logic to the router) and verified with a full create → read → delete → read sequence returning 200 / 200 / 404 as expected.

Verified endpoint status after repair

Endpoint	Result
POST /entries	Working
GET /entries	Working
GET /entries/{id}	Working (repaired — was a 501 stub)
PATCH /entries/{id}	Working
DELETE /entries/{id}	Working (repaired — was a 501 stub)
DELETE /entries	Working

5.3 Phase 3 — Containerize and Push to ACR

A multi-stage Dockerfile was written: a build stage installs Python dependencies, and a slim runtime stage copies only the installed packages and application code, running as a non-root user with a health check against the application root path.

```
FROM python:3.12-slim AS builder
WORKDIR /app
COPY api/requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
RUN useradd -m appuser \
  && apt-get update && apt-get upgrade -y && rm -rf /var/lib/apt/lists/*
COPY --from=builder /root/.local /home/appuser/.local
COPY api/ .
ENV PATH=/home/appuser/.local/bin:$PATH
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:8000/ || exit 1
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The image was built, tested locally via docker compose against a disposable PostgreSQL container, scanned with Trivy, and pushed to Azure Container Registry (m3p2acr).

Vulnerability scan result

The initial Trivy scan reported 159 findings across all severities (44 High, 53 Medium, 57 Low, 5 Unknown, 0 Critical). Investigation confirmed the High-severity findings are Debian 13 base-OS packages (util-linux, systemd, glibc, ncurses, perl) with no vendor-published fix at the time of scanning — confirmed unchanged even after an explicit apt-get upgrade in the final image stage. With 0 Critical and no actionable fix available for the High findings, this result was accepted as the project's vulnerability baseline and used consistently as the pass/fail bar for every subsequent scan in this project, including the CI/CD pipeline gate (Section 5.7).

5.4 Phase 4 — Provision Infrastructure with Terraform

All required infrastructure was defined in Terraform and applied to the module3-project2 resource group in West Europe: a resource group, virtual network with two subnets (AKS and database), an AKS cluster, a PostgreSQL Flexible Server with a private DNS zone, Azure Container Registry, and Azure Key Vault.

Final resource configuration

Resource	Key Configuration
AKS Cluster	learningsteps-aks — 2 x Standard_D2s_v6 nodes, RBAC enabled, Azure CNI network policy, OIDC issuer enabled
PostgreSQL Flexible Server	learningsteps-db-m3p2acr — VNet-integrated, public access disabled, private DNS zone linked
Container Registry	m3p2acr — Basic SKU
Key Vault	learningsteps-kv-m3p2acr — purge protection enabled, network ACL default-deny with AKS subnet and admin IP allow-listed

Infrastructure security scan

trivy config against the Terraform configuration initially reported six misconfigurations (two Critical, two High, two Medium). Four were remediated directly in code: AKS RBAC was enabled, an Azure CNI network policy was applied, Key Vault network ACLs were set to default-deny, and Key Vault purge protection was enabled — reducing the result to three remaining findings (one Critical: AKS API server IP restriction; two Medium: AKS diagnostic logging and Key Vault soft-delete retention tuning). These three were consciously left open as a documented, accepted trade-off rather than implemented, since fixing them required information and resources (a static admin IP allow-list, a Log Analytics workspace) outside the project's core scope.

Three genuine Azure platform constraints were also resolved during provisioning — a mandatory Private DNS zone for VNet-integrated PostgreSQL, an AKS service CIDR collision with the VNet's own address range, and a subscription vCPU quota limit on the originally planned VM size — each documented in full in Section 7.

5.5 Phase 5 — Connect Key Vault Secrets

The PostgreSQL connection string was stored as a secret in Key Vault. The AKS Key Vault Secrets Provider add-on was enabled, and its managed identity was granted read access to the secret. A SecretProviderClass Kubernetes resource was created, mounting the secret into the application pod and projecting it as the DATABASE_URL environment variable via a synced Kubernetes Secret — the application code itself requires no Key Vault awareness.

End-to-end delivery was verified: the running pod's environment was inspected directly to confirm DATABASE_URL was populated from Key Vault, and a live POST to /entries against the Azure-hosted PostgreSQL instance succeeded, proving the full chain — Key Vault, CSI driver, pod environment, application, database — functions correctly.

5.6 Phase 6 — Deploy with Helm Chart

The flat Kubernetes manifests used for Phase 5 verification were replaced with a parameterized Helm chart (Chart.yaml, values.yaml, and templated Deployment, Service, HorizontalPodAutoscaler, and SecretProviderClass resources), giving the application a single versioned, upgradeable release rather than a set of loose YAML files.

Because Helm refuses to adopt Kubernetes resources it did not itself create, the Phase 5 manifests were removed and the application re-created cleanly under Helm management — a zero-data-loss operation, since the removed resources were stateless application configuration, not the database.

Checkpoint verification

- Self-healing confirmed: deleting the running pod resulted in an automatic, differently-named replacement pod.
- The HorizontalPodAutoscaler was confirmed active and reading real CPU metrics from the running pod.
- The Service was confirmed to have a public LoadBalancer IP.
- A live end-to-end test against the public IP (POST /entries) returned HTTP 200 with a correctly persisted record.

5.7 Phase 7 — CI/CD Pipeline (OIDC + Security Gates)

A GitHub Actions workflow was built implementing the full build–scan–deploy pipeline. The project repository was forked to the author's own GitHub account (Hany-Dh/learningsteps) rather than modifying the original upstream CyberstepsDE repository, and all pipeline work targets this fork.



Figure 3 — CI/CD pipeline stages, in execution order.

Pipeline stage summary

Stage	Tool	Gate Behavior
Secret scan	trufflehog	Any finding fails the pipeline — no severity threshold
Lint and test	ruff, pytest	Non-blocking, informational
Build image	docker build	Must succeed to proceed
Image vulnerability scan	Trivy	Fails the pipeline on any CRITICAL finding
Azure authentication	OIDC via azure/login	Federated identity — no stored Azure credential
Push to registry	az acr / docker push	Pushes only the image that passed scanning
Deploy	helm upgrade --install	Deploys the exact scanned image by immutable tag

Authentication uses OpenID Connect federation between GitHub Actions and an Azure AD application registration scoped to this repository and branch, with Contributor role assigned on the resource group — no Azure password or long-lived secret is stored in GitHub at any point.

A full pipeline run was confirmed green end to end: secret scan, lint/test, build, vulnerability scan, OIDC login, registry push, and Helm deploy all succeeded in sequence, with the pipeline-deployed pod and public endpoint independently re-verified afterward.

The image vulnerability gate was deliberately scoped to CRITICAL severity only, matching the baseline already established and accepted in Phase 3 (Section 5.3) — the same 44 High-severity, unfixed-upstream findings would otherwise block every single pipeline run indefinitely with no actionable remediation available.

A Terraform scan step was planned for this workflow but removed: the infra-terraform directory is intentionally kept outside the application's git repository (infrastructure and application delivery are owned by separate tools, per the project's own architectural principle), so it is not present in the pipeline's checkout and cannot be scanned there without merging two deliberately separate concerns.

Seven real issues were encountered and resolved while building this pipeline — repository ownership, authentication, a malformed scan flag, gate severity tuning, the Terraform-scan scope mismatch, an OIDC subject-claim format specific to forked repositories, and a missing chart directory in the repository. Full detail is in Section 7.

5.8 Phase 8 — Optional Monitoring (Prometheus & Grafana)

This phase is explicitly marked optional in the project brief. Work proceeded far enough to prove the infrastructure-level goal, then was deliberately paused in favor of completing the mandatory Phase 9 handover items.

Completed

- The FastAPI application was instrumented with prometheus-client and prometheus-fastapi-instrumentator, exposing a live /metrics endpoint.
- Prometheus and Grafana were deployed via the kube-prometheus-stack Helm chart, using a deliberately slimmed configuration (Alertmanager disabled, no persistent storage, reduced resource requests) sized to the cluster's real available capacity.
- A Kubernetes ServiceMonitor was created, and the application's Service was labeled and given a named port for Prometheus service-discovery; the scrape target was independently confirmed healthy ("up", no errors) via the Prometheus API.
- Grafana login and its pre-provisioned Prometheus data source were both confirmed working.

Remaining (not completed)

- A prebuilt community dashboard import was attempted and abandoned — it required a Loki log data source that is not part of this deployment.
- A manual three-panel dashboard (HTTP request volume, API latency, application/database reachability) was begun but not completed or saved.
- A live-traffic dashboard verification was not performed.

Resource sizing decision

Given the AKS node pool's modest size (2 x Standard_D2s_v6 — 4 vCPU / 16 GiB total), the monitoring stack was deliberately installed without Alertmanager and without persistent storage, since the project's checkpoint requires only a live dashboard reacting to traffic, not alert routing or long-term metric retention. Measured headroom before installation (7%/4% actual CPU utilization) confirmed ample capacity for the slimmed install.

5.9 Phase 9 — Final Demonstration & Handover

Of the four handover items defined in Section 2.3, two have already been demonstrated live during development (Continuous Deployment, in Section 5.7; the optional Visual Insights item, substantially in Section 5.8). The remaining two — Security Enforcement and Infrastructure Recovery — are documented in full, ready-to-execute form in Section 10, to be performed live during project presentation rather than against the working demonstration environment in advance, since both are inherently destructive to the current running state (a deliberately broken dependency, and a full infrastructure teardown).

6. Feature Enhancement — Dual Content-Type Support

Beyond the baseline project requirements, the application's write endpoints were extended to accept either application/json or text/plain request bodies, broadening client compatibility (for example, simple shell scripts or tools that cannot easily construct JSON) without any change to the underlying data model, service layer, or database logic.

Implementation

A shared `parse_request_body()` helper inspects the incoming Content-Type header: JSON bodies are parsed and validated as before; plain-text bodies are accepted as free text and mapped to the entry's work field, with the struggle and intention fields made optional in the data model to support this narrower text-only case. Both the `POST /entries` and `PATCH /entries/{id}` endpoints were updated to use this helper.

An `openapi_extra` schema was added to both endpoints' route declarations. This was necessary because FastAPI cannot auto-generate interactive API documentation for an endpoint whose request body is read as a raw Request object rather than a typed Pydantic model — without it, the Swagger UI at `/docs` silently continued to display only the JSON option even though text/plain was already functionally accepted, which was the initial point of confusion during testing.

Verification

Test	Method	Result
POST, application/json	curl with JSON body	201 — entry created
POST, text/plain	curl with raw text body	201 — entry created, text mapped to work field
PATCH, application/json	curl with JSON body	200 — entry updated
PATCH, text/plain	curl with raw text body	200 — entry updated
OpenAPI schema	GET /openapi.json	Both content types listed for POST and PATCH

All four combinations were verified live against the production Azure PostgreSQL instance, and the change was delivered through the standard CI/CD pipeline (Section 5.7) with no manual deployment steps.

7. Consolidated Troubleshooting and Issue Log

Every environment- or platform-level issue encountered over the course of the project is recorded below, in the order encountered, with its root cause and resolution. This log reflects genuine engineering problem-solving rather than a sanitized happy-path account, and is retained as evidence of practical troubleshooting competency.

Issue	Root Cause	Resolution
Azure CLI apt install failed on Kali	Kali's rolling-release codename has no matching Microsoft apt package	Installed via pipx instead of apt
tfsec not found after install	tfsec was deprecated and merged into Trivy	Switched all Terraform scanning to trivy config
Docker permission denied	Shell session lacked the docker group / stale DOCKER_HOST	usermod -aG docker + newgrp; unset DOCKER_HOST
GET/DELETE /entries/{id} returned 501	Intentional stub endpoints in exercise code	Wired existing service-layer methods to the router
Dockerfile not found by compose	File saved as dockerfile.txt instead of Dockerfile	Renamed to exact Dockerfile
Postgres container failed to bind port 5432	A native Postgres service was already running on Kali	Remapped container to host port 5433
App container 500 on DB write	DATABASE_URL pointed at localhost instead of the compose service name	Corrected to the internal service hostname
az acr / az acr login hung indefinitely	Resource group was never created before az acr create	Ran az group create first
terraform apply: resource already exists	ACR, resource group, and VNet had been created manually in earlier phases	Imported each into Terraform state
terraform apply: polling status Unknown	Transient Azure API glitch (VNet, then AKS)	Verified via az ... show; resource had actually succeeded
Postgres: EmptyPrivateDnsZoneArmResourceId	VNet-integrated Postgres requires a Private DNS zone	Added azurerm_private_dns_zone + VNet link
AKS: VM size not allowed on subscription	Standard_B2s not in the subscription's allowed SKU list	Switched to an allowed size, later Standard_D2s_v6
Postgres: conflicting public access + VNet config	Azure rejects public access alongside VNet integration	Set public_network_access_enabled = false
AKS: ServiceCidrOverlapExistingSubnetsCidr	Default AKS service CIDR collided with the VNet's own range	Set explicit non-overlapping service_cidr
AKS: insufficient vCPU quota	Subscription quota for the VM family was exhausted	Switched VM family (D-series), restoring 2-node count
Key Vault secret set: Forbidden	Own account lacked secret-set permission; then Key Vault firewall blocked the client IP	Granted access policy; added IP network rule

CSI mount failed: nodePublishSecretRef	SecretProviderClass missing identity parameters	Added usePodIdentity / useVMManagedIdentity / userAssignedIdentityID
Pod ImagePullBackOff	AKS kubelet identity lacked ACR pull rights	az aks update --attach-acr
Helm install refused to adopt existing resources	Raw kubectl-applied resources lack Helm ownership labels	Deleted and recreated cleanly under Helm
AKS / Postgres periodically found Stopped	Subscription-level auto-stop after inactivity	az aks start / az postgres flexible-server start; checked routinely each session
GitHub push: Invalid username or token	GitHub removed password authentication for git operations	Used a Personal Access Token
trufflehog: flag 'fail' cannot be repeated	Action already passes --fail by default; duplicated by extra_args	Removed the redundant extra_args
Pipeline failed Trivy gate at 44 High / 0 Critical	Gate was set to HIGH,CRITICAL against an already-accepted baseline	Narrowed the gate to CRITICAL only, matching Phase 3's bar
Pipeline: infra-terraform not found	Terraform directory is outside the application's git repository by design	Removed the Terraform scan step from this workflow
OIDC login: no matching federated identity record	Forked-repo token subject uses an @-separated owner ID, not the plain form	Recreated the federated credential with the exact subject from the error
Helm deploy: chart path not found	Chart directory existed locally but was never committed to the application repository	Copied the chart into the repo and committed
ServiceMonitor found no targets	Service lacked a named port and label for discovery	Added a named port and matching label to the Service

8. Security Posture Summary

8.1 Application Image

Final Trivy image scan: 159 total findings (0 Critical, 44 High, 53 Medium, 57 Low, 5 Unknown). All High-severity findings are Debian 13 base-OS packages with no vendor fix available at scan time; this is treated as the project's accepted baseline, consistent across every phase of the project and the CI/CD gate itself.

8.2 Infrastructure (Terraform)

Final trivy config scan: 3 remaining findings (1 Critical, 2 Medium), down from an initial 6, after remediating AKS RBAC, AKS network policy, and Key Vault network ACLs / purge protection directly in code. The remaining findings (AKS API server IP allow-listing, AKS diagnostic logging, Key Vault soft-delete retention tuning) are documented, accepted trade-offs outside core project scope.

8.3 Secrets Management

- No application secret is stored in source control at any point.
- The database connection string lives exclusively in Azure Key Vault and is delivered to the running pod via the CSI Secrets Store driver.
- Key Vault network access defaults to deny, with explicit allow rules for the AKS subnet and the administrator's own IP.
- GitHub Actions authenticates to Azure via OIDC federation — no Azure credential of any kind is stored as a GitHub secret.

8.4 CI/CD Pipeline Gates

- Secret scanning (trufflehog): any finding blocks the pipeline unconditionally.
- Image vulnerability scanning (Trivy): blocks on any CRITICAL finding.
- The deployed image is always the exact, immutably-tagged image that passed scanning — no rebuild occurs between scan and deploy.

9. Observability Status

Metrics collection is live and independently verified: the application exposes Prometheus-format metrics, Prometheus scrapes them successfully (confirmed via direct API query), and Grafana is connected to Prometheus as a working data source. Dashboard visualization was not completed and is the only outstanding item from the project's optional scope; see Section 5.8 for full detail.

10. Final Demonstration Plan

The following two procedures satisfy the two remaining required handover criteria (Section 2.3). Both are fully specified and ready to execute; they are deliberately reserved for live presentation rather than run in advance, because each is destructive to the current working demonstration environment — the first intentionally breaks the pipeline, and the second tears down and rebuilds all Azure infrastructure, which would discard the live public endpoint and any data not otherwise exported.

10.1 Security Enforcement — Prove the Pipeline Blocks a Bad Push

#	Step	Command
1	Create a test branch	<code>git checkout -b security-test</code>
2	Add a known-vulnerable dependency	Add <code>pyyaml==5.3.1</code> to <code>api/requirements.txt</code>
3	Commit and push the branch	<code>git add -A && git commit -m "Test: vulnerable dependency" && git push</code>
4	Push to main to trigger the pipeline	<code>git push origin security-test:main</code>
5	Observe the pipeline fail at the Trivy gate	GitHub Actions tab — capture the failing step and finding
6	Revert the change	<code>git revert HEAD && git push origin main</code>
7	Confirm the pipeline passes again	GitHub Actions tab — green run

10.2 Infrastructure Recovery — terraform destroy → terraform apply

#	Step	Command
1	Confirm state matches reality	<code>terraform plan</code> (expect no changes)
2	Record current outputs for comparison	<code>terraform output</code>
3	Destroy all managed infrastructure	<code>terraform destroy</code>
4	Confirm the resource group is empty	<code>az resource list --resource-group module3-project2</code>
5	Re-apply from code only	<code>terraform apply</code>
6	Re-attach ACR pull rights to AKS	<code>az aks update --attach-acr m3p2acr</code>
7	Re-create the Key Vault secret and network rules	<code>az keyvault secret set / network-rule add</code> (Section 5.5)
8	Re-deploy the application	<code>helm upgrade --install learningsteps ./learningsteps-chart</code>
9	Verify the public endpoint is live	<code>curl</code> against the new public IP /docs

Caution: Step 10.2 destroys the live public IP address and any PostgreSQL data not otherwise exported. Execute only when ready to rebuild the demonstration environment from code.

11. Conclusion

All nine required phases of the LearningSteps Evolution project were completed and independently verified against a live Azure environment: the application is containerized and scanned, infrastructure is fully codified in Terraform, secrets are managed through Key Vault with no credential ever committed to source control, the application runs as a self-healing, autoscaling Kubernetes workload delivered through a versioned Helm release, and every code change is built, security-scanned, and deployed automatically through a GitHub Actions pipeline authenticated via passwordless OIDC federation.

The optional observability phase was substantially implemented at the infrastructure level before being deliberately paused, and an unplanned API enhancement — dual JSON/plain-text request support — was designed, implemented, and verified end to end during the project. The two remaining handover demonstrations are fully planned and ready for live execution.

Over the course of the project, more than twenty distinct real-world technical issues were encountered across local tooling, Azure platform constraints, Kubernetes networking, Helm resource ownership, and CI/CD authentication — each diagnosed from first principles and resolved, with full detail retained in Section 7. This troubleshooting record, as much as the finished system itself, reflects the practical engineering competency the project was designed to build.

Appendix A — Final Terraform Configuration (main.tf, abridged)

```
resource "azurerm_kubernetes_cluster" "main" {
  name                = "learningsteps-aks"
  dns_prefix          = "learningsteps"
  role_based_access_control_enabled = true
  default_node_pool {
    name       = "default"
    node_count = 2
    vm_size    = "Standard_D2s_v6"
  }
  network_profile {
    network_plugin = "azure"
    network_policy = "azure"
    service_cidr   = "10.1.0.0/16"
    dns_service_ip = "10.1.0.10"
  }
  identity { type = "SystemAssigned" }
}

resource "azurerm_postgresql_flexible_server" "main" {
  name                = "learningsteps-db-m3p2acr"
  version             = "16"
  delegated_subnet_id = azurerm_subnet.db.id
  private_dns_zone_id = azurerm_private_dns_zone.postgres.id
  public_network_access_enabled = false
  sku_name            = "B_Standard_B1ms"
  storage_mb          = 32768
}

resource "azurerm_key_vault" "main" {
  name                = "learningsteps-kv-m3p2acr"
  purge_protection_enabled = true
  network_acls {
    default_action = "Deny"
    bypass         = "AzureServices"
  }
}
```

Appendix B — GitHub Actions Workflow (ci-cd.yml, abridged)

```

name: build-scan-deploy
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read
jobs:
  build-and-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - name: Scan for secrets
        uses: trufflesecurity/trufflehog@main
      - name: Build image
        run: docker build -t learningsteps:${{ github.sha }} .
      - name: Scan image for vulnerabilities
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: learningsteps:${{ github.sha }}
          severity: CRITICAL
          exit-code: '1'
      - name: Azure login (OIDC)
        uses: azure/login@v3
        with:
          client-id: ${ secrets.AZURE_CLIENT_ID }
          tenant-id: ${ secrets.AZURE_TENANT_ID }
          subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
      - name: Push to ACR
        run: |
          az acr login --name m3p2acr
          docker push m3p2acr.azurecr.io/learningsteps:${{ github.sha }}
      - name: Deploy with Helm
        run: |
          az aks get-credentials --name learningsteps-aks --resource-group module3-project2
          helm upgrade --install learningsteps ./learningsteps-chart --set image.tag=${{
github.sha }}

```

Appendix C — Helm Chart Structure

```

learningsteps-chart/
  Chart.yaml
  values.yaml
  templates/
    _helpers.tpl
    deployment.yaml
    service.yaml
    hpa.yaml
    secretproviderclass.yaml

```

Appendix D — Tool and Platform Reference

Category	Tool / Resource	Identifier
Source control	GitHub repository	github.com/Hany-Dh/learningsteps
Container registry	Azure Container Registry	m3p2acr
Orchestration	Azure Kubernetes Service	learningsteps-aks
Database	PostgreSQL Flexible Server	learningsteps-db-m3p2acr
Secrets	Azure Key Vault	learningsteps-kv-m3p2acr
Resource Group	Azure Resource Group	module3-project2
Region	Azure Region	West Europe
Scanning	Trivy	Image + Infrastructure-as-Code scanning
Secret scanning	trufflehog	Repository and commit history scanning
IaC	Terraform	AzureRM provider
Package manager	Helm	kube-prometheus-stack, application chart

Appendix E — Project Progress Tracker (Summary)

Step	Description	Status
0	Plan & architecture	Done
1	Install tools on Kali	Done
2	Get the app working locally	Done
3	Dockerize + push to ACR	Done
4	Terraform infrastructure	Done
5	Connect Key Vault secrets	Done
6	Deploy with Helm chart	Done
7	CI/CD pipeline (OIDC + gates)	Done
8	Optional monitoring	Paused (optional)
9	Final demo	Plan documented — Section 10

Index of Figures and Tables

Figure 1 — End-to-end system architecture Section 3

Figure 2 — Ten project phases, execution order Section 4

Figure 3 — CI/CD pipeline stages Section 5.7

Table — Success criteria status Section 2.3

Table — Component summary Section 3.1

Table — Pipeline stage summary Section 5.7

Table — Consolidated troubleshooting log Section 7

Table — Security enforcement procedure Section 10.1

Table — Infrastructure recovery procedure Section 10.2

